Told Nothing, Forgetting Little: Boundary-Blind Continual Reinforcement Learning with Competence-Gated Memory and Surprise Resets

Anonymous Authors

***Abstract*—Continual reinforcement learning agents are usually told when a task ends, and use that signal to consolidate knowledge, store data or reset parts of the network. Children and animals get no such signal. We study an agent that is never told about task changes and uses only its own success. It (i) stores observations and its action probabilities in a fixed-size reservoir only while it is competent, (ii) distils its policy toward this memory on every update, and (iii) when its success collapses after mastery, treats the surprise as a change of the world and re-initialises its whole network, so that old skills return only through memory. With PPO and 20 seeds, on an 8-stage MiniGrid sequence with six kinds of task the agent keeps 92.7% success, on par with the same learner told the boundaries and 24.9 points above CLEAR. On five MinAtar games in sequence it keeps 44.4% of single-game scores, against 16.2% for CLEAR, 5.6% without protection and 32.8% for joint training, and on par with Reset & Distill, which is told every boundary. With the same memory, the self-triggered reset beats a reset at every given boundary by 6.6 points (20 of 20 seeds). All claims were replicated on 10 new seeds under a rule fixed in advance.**

***Index Terms*—continual learning, reinforcement learning, catastrophic forgetting, loss of plasticity, task-free learning.**

I. Introduction

A child is not told that a new task starts now. It notices that what used to work fails, adapts, and keeps what it once mastered. Most continual reinforcement learning (RL) methods [1] assume the opposite: task identities or boundaries are given, and they drive consolidation [2], [3], per-task buffers, or network resets [4]. Task-free methods exist [5], [6], [7], [8], [9]; in RL, CLEAR [10] replays all past experience with behavioural cloning and needs no labels. Separately, networks that keep training lose the ability to learn new things [11], [12], [13], [14], [15] [16].

We ask how far an agent can get when it must decide by itself *what* to remember and *when* the world has changed, with one fixed-size network and a fixed-size memory. Our answer is a small loop driven only by the agent's own success: remember what you have mastered, distil toward that memory, and restart the network when the world surprises you.

Our contributions are: (1) a boundary-blind learner built from three self-triggered parts, namely competence-gated reservoir memory, replay distillation, and a surprise-triggered full reset (Sec. III); (2) evidence on two domains, MiniGrid (3-task and 8-stage sequences) and five MinAtar games, with 20 seeds each, against CLEAR, no protection, joint training, and two methods that are told the boundaries (Sec. V); (3) analyses showing that memory, not the reset, keeps old skills; that the reset, not dormant-unit repair, restores learning on MinAtar; and that detecting change beats being told when not every change needs a reset (Sec. VI).

All hyperparameters were chosen on separate tuning seeds (101–105). Final seeds 1–10 were run once; seeds 11–20 were then run as a replication with a rule fixed beforehand: a claim stays only if seeds 11–20 alone point the same way. One secondary claim was weakened by this rule, and we report it.

II. Related Work

Continual RL. Networks trained on a sequence of tasks forget earlier ones. Khetarpal et al. [1] review continual RL; Wołczyk et al. [17] show that forward transfer, not only forgetting, is a key open problem. *Regularisation* slows change of important weights, e.g., EWC [2]; *distillation* keeps the outputs of an earlier model, e.g., Learning without Forgetting [18], policy distillation [19] and Progress & Compress [3]; *replay* re-trains on stored experience [20]. CLEAR [10] combines off-policy learning and behavioural cloning from replay with on-policy learning and needs no task identities; like our agent, it is task-free, so it is our main baseline [16].

Task-free continual learning. Aljundi et al. [5] consolidate at plateaus of the loss, without task boundaries; our rule "store only while competent" is the same spirit, in RL and with success instead of loss. CURL [6] infers tasks inside the model. In RL, OWL [7] keeps one head per task and selects a head with a bandit; 3RL [8] adds a recurrent context to SAC with replay and finds that task-agnostic learning can beat multi-task training; Dick et al. [9] detect context changes with statistical tests and train one policy per detected context. Our agent keeps a single fixed-size network, detects a change from a collapse of its own success, and gates which experience enters a reservoir [21], a strong replay memory in supervised continual learning [22].

Loss of plasticity. Networks that keep training lose plasticity [11], [12], [13]; Sokar et al. [14] relate it to dormant neurons and re-initialise them (ReDo). Nikishin et al. [15] reset the last layers, or the whole network, on a fixed schedule in single-task agents; Ash and Adams [23] shrink and perturb weights. Reset & Distill [4] is the closest recipe to ours: it resets actor and critic after each task and distils old experts from small stored buffers, but it is told when each task ends. Our reset is instead triggered by self-detected surprise, and old skills return only through distillation from the competence-gated memory.

III. Method

Setting. One agent faces tasks T1, T2, … in sequence, each for a fixed number of environment steps. It receives

observations and rewards only; it is never given task identity, boundaries, or the number of tasks [16].

Base learner. PPO [24] in a CleanRL-style implementation [25]: 16 parallel environments, 128-step rollouts, 4 epochs, 8 minibatches, Adam with learning rate 10^{-3}, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2, entropy coefficient 0.05, value coefficient 0.5, gradient-norm clip 0.5. A small CNN body ends in a 64-unit tanh layer, followed by linear actor and critic heads. The same settings are used for every method and both domains.

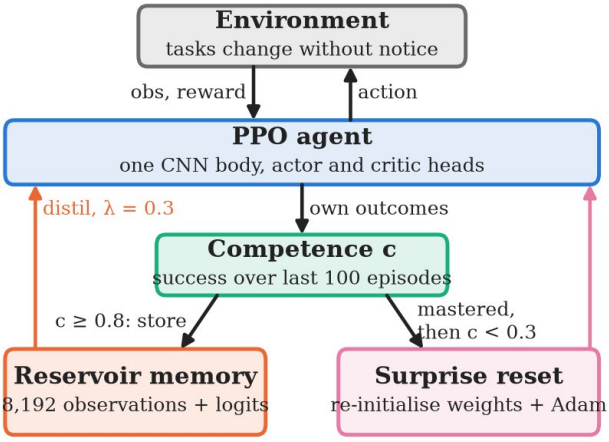


Fig. 1. The boundary-blind loop. Every trigger comes from the agent's own success c; no task identity or boundary is given. The reservoir only receives behaviour recorded while $c \geq 0.8$. A surprise ($c < 0.3$ after mastery) re-initialises the network and optimiser; old skills then return only through distillation.

Competence. c is the mean outcome of the agent's last 100 finished episodes. In MiniGrid the outcome is success (reward > 0). MinAtar has scores rather than success, so c is the mean score of the last 100 episodes divided by the best such mean since the last surprise: competence is measured relative to the agent's own best, never to an external target.

(i) Competence-gated memory. While $c \geq m$ (mastery, $m = 0.8$), after each rollout 256 random observations of that rollout are offered, with the current policy's log-probabilities, to a reservoir of fixed size $N = 8192$ using reservoir sampling [21]. The memory therefore holds a uniform sample of all *mastered* behaviour, never grows, and stores no rewards or actions.

(ii) Replay distillation. Every PPO minibatch adds $\lambda \cdot \mathrm{KL}(\pi_{\text{stored}} \,\|\, \pi_\theta)$, averaged over 256 reservoir observations, with $\lambda = 0.3$. Old behaviour is kept as a function of observations, not as frozen weights.

(iii) Surprise reset. If the agent had mastered ($c \geq m$) and c then drops below 0.3, it logs a surprise, clears the mastered flag, and re-initialises all network parameters and the Adam state. It keeps the reservoir; from the next update, distillation pulls the fresh network back toward the stored behaviour while PPO learns the new task.

Why a full reset. On MiniGrid the plasticity loss is a near-deterministic policy, and resetting only the output heads cures it. On MinAtar the entropy stays high, yet later games are learned at about 59% of scratch; only a full reset restores this (Sec. VI-B). We use the full reset everywhere; on MiniGrid it gives the same results as the head reset.

IV. Experimental Setup

MiniGrid [26]. *C1, 3 tasks:* Empty → DoorKey → LavaGap, each a mix of three sizes, 600k steps per task; 12 tests including unseen sizes, 50 episodes each. *L1, 8 stages, 6 task kinds:* Empty, DoorKey, LavaGap, Dynamic-Obstacles, SimpleCrossing, Unlock, DoorKey (revisit), LavaGap (revisit), 500k steps per stage; 11 tests of trained kinds and sizes plus KeyCorridor (never trained), 30 episodes each. The metric is the mean final success over the trained tests [16].

MinAtar [27]. Breakout → Space Invaders → Freeway → Asterix → Seaquest, 2M steps each, sticky actions 0.1. Each score is divided by the mean score of a single-game learner trained for 2M steps from scratch (10 seeds); 30 evaluation episodes per game. The metric is the mean over games after the whole sequence; "just learned" is the same ratio measured right after each game's own stage.

Baselines. *No protection:* plain sequential PPO. *CLEAR* [10], adapted to PPO: a reservoir of all past experience (same size, 8192), an off-policy clipped importance-weighted policy gradient on replay, plus coefficient × (policy KL + value MSE) cloning; coefficient 1.0 chosen on MiniGrid tuning seeds (0.1 is also reported for MinAtar). It is our adaptation, not the original IMPALA implementation. *Joint:* all tasks mixed, same total steps (MinAtar also 2×). *Told distillation:* told each task end; it then stores the finished task's observations, resets heads and Adam, and distils toward them (KL, 0.3). *Reset & Distill* [4], adapted to PPO: told each task end; full reset plus distillation toward the last rollout of every finished task. *Told reset + our memory:* our competence-gated reservoir and distillation, but a full reset at every given boundary instead of on surprise.

Protocol. Tuning seeds 101–105; final seeds 1–20 (Sec. I). We report mean ± 95% confidence interval (t-distribution) and paired differences over seeds with the number of seeds where ours is higher. Analyses in Sec. VI use seeds 1–10 unless stated. Every result file stores its configuration and code commit; runs are bit-exact per seed on the final code, and six baseline groups whose first ten seeds predate version control were re-run before pooling. All runs used one 12-thread CPU.

V. Results

A. MiniGrid

TABLE I
Final success on MiniGrid (%, 20 seeds, mean ± 95% CI)

Method	Told	L1 (8 stages)	C1 (3 tasks)	C1 unseen sizes
No protection	–	32.9 ± 4.5	37.8 ± 8.0	32.2 ± 9.1
CLEAR [10]	no	67.8 ± 1.7	92.3 ± 1.7	86.9 ± 2.6
Joint (all tasks mixed)	–	86.2 ± 4.6	95.2 ± 1.8	91.2 ± 2.8
Told distillation	yes	90.8 ± 3.0	92.5 ± 5.2	88.7 ± 5.7
Reset & Distill [4]	yes	87.1 ± 5.3	89.4 ± 5.4	83.3 ± 6.2
Told reset + our memory	yes	86.1 ± 1.4	96.3 ± 1.1	92.9 ± 2.0
Ours (blind)	**no**	**92.7 ± 0.9**	**96.4 ± 0.8**	**93.1 ± 1.7**

On the 8-stage sequence L1 (Table I, Fig. 2), the agent that is never told anything keeps 92.7% (worst seed 89), on par with told distillation (+1.9 ± 3.3), and above CLEAR (+24.9 ± 2.1, 20/20 seeds), joint training (+6.5 ± 4.9, 17/20) and no protection (+59.7 ± 4.6, 20/20). CLEAR keeps early tasks but learns the new kinds (Obstacles, Crossing) poorly: replaying everything holds the policy to old behaviour. KeyCorridor, never trained, stays at 0% for all methods: there is no zero-shot composition [16].

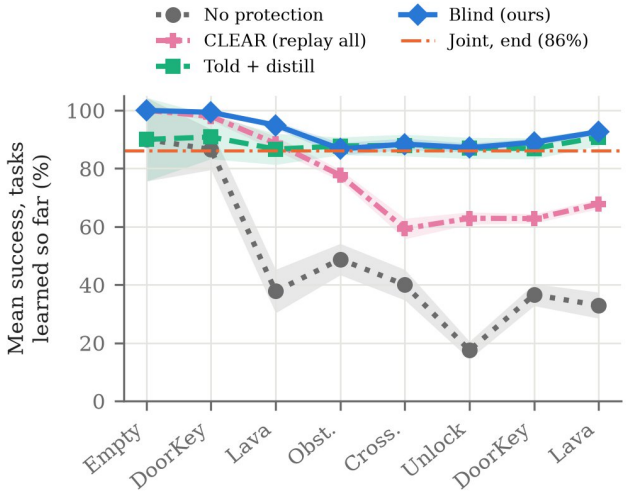


Fig. 2. MiniGrid L1: mean success on all task kinds learned so far, after each of the 8 stages (mean and 95% CI, 20 seeds). The dash-dotted line is joint training on all tasks for the same total steps, at its end.

On the smaller 3-task sequence C1, ours reaches 96.4% (93.1% on unseen sizes), above CLEAR (+4.1 ± 1.8, 16/20) and on par with joint training (+1.1 ± 2.0) and told distillation (+3.9 ± 5.3). Here the differences to the strong baselines are small.

B. MinAtar

TABLE II
MINATAR AFTER ALL FIVE GAMES (% OF SCRATCH, 20 SEEDS)

Method	Final	Just learned	Ours − method (seeds)
No protection	5.6 ± 0.9	67.5 ± 2.5	+38.7 ± 2.3 (20/20)
CLEAR, coef 1.0	16.2 ± 0.5	22.2 ± 0.8	+28.2 ± 2.1 (20/20)
CLEAR, coef 0.1*	21.4 ± 2.0	42.5 ± 5.9	+21.3 ± 3.5 (10/10)
Joint, 10M steps	32.8 ± 2.8	–	+11.6 ± 3.4 (19/20)
Joint, 20M steps*	36.2 ± 3.7	–	+6.5 ± 4.2 (8/10)
Reset & Distill (told)	45.1 ± 2.5	95.1 ± 4.0	−0.7 ± 3.2 (10/20)
Ours (blind)	**44.4 ± 2.0**	**92.0 ± 3.1**	–

Seeds 1–10. Coefficient 0.1 was the best of three CLEAR coefficients on MinAtar tuning seeds

The MiniGrid settings were reused; nothing method-specific was tuned on MinAtar except choosing the full reset by a rule fixed in advance (Sec. VI-B). Ours keeps 44.4% of single-game scores after all five games (Table II, Fig. 3), above CLEAR at its best coefficient, no protection and our joint baseline, and on par with Reset & Distill, which is told every boundary. The surprise fired within 500k steps of every game switch (all 4 switches, 10/10 seeds). CLEAR's weakness here is plasticity, not memory: it learns new games at only 22% of scratch. Joint training is not an upper bound on MinAtar: even with 20M steps it never learns Freeway, Asterix or Seaquest, consistent with the interference reported for multi-task SAC [8]; we therefore claim "above our joint baseline", not "above joint training". Retention is partial: games learned first keep the least.

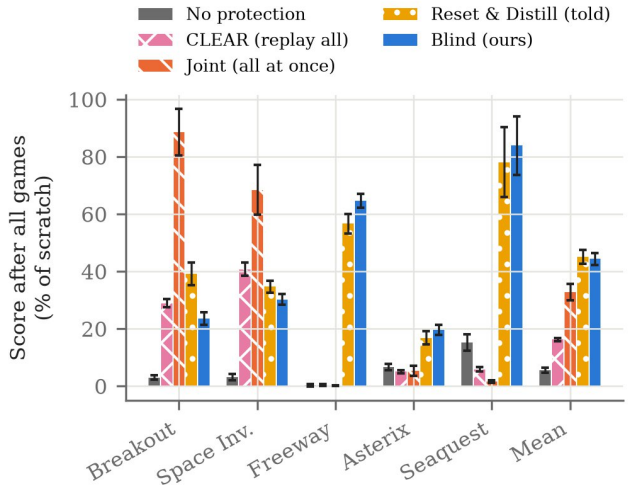


Fig. 3. MinAtar: score on each game after all five games, as % of a single-game learner trained for 2M steps (mean and 95% CI, 20 seeds).

C. Replication on New Seeds

All claims above held on seeds 11–20 alone, for example ours − CLEAR +30.1 ± 3.6 on MinAtar and +23.8 ± 2.6 on L1 (both 10/10), and ties with the told methods. One claim was weakened (Sec. VI-C).

VI. ANALYSIS

TABLE III
ANALYSES (SEEDS 1–10 UNLESS STATED)

Variant	Score	Change vs. ours
No reservoir, MinAtar	25.3	−17.4 ± 5.3 (10/10)
No reservoir, MiniGrid C1	39.0	−57.1 ± 9.7
Head reset only, MinAtar	28.1 ± 1.0	−14.6 ± 2.5 (10/10)
Told reset + our memory, L1 (20 seeds)	86.1 ± 1.4	−6.6 ± 1.4 (20/20)
Reversed game order, MinAtar	37.2 ± 2.1	CLEAR 24.6, none 20.0
Gradual change, MiniGrid C1	97.0 ± 1.4	abrupt 96.1 ± 1.7
Gradual change, MinAtar	42.5 ± 4.5	abrupt 42.7 ± 2.2

A. Memory, Not the Reset, Keeps Old Skills

Without the reservoir, the full reset learns each new MinAtar game at 105% of scratch, but the final score falls to 25.3% (Table III); on MiniGrid C1 it falls to 39.0%. The reset brings back the ability to learn; the memory carries the past [16].

B. Two Kinds of Plasticity Loss

On MiniGrid C1 without protection, learning the third task tracks the policy entropy at its start: with entropy 0.01 (of 1.95) 0/10 seeds learn it; resetting the output heads restores entropy to 1.83 and learning on 10/10 seeds (Fig. 4). On MinAtar, entropy is not collapsed (1.2–1.4 of 1.79), dormant units stay

below 2%, and ReDo [14] every 10 rollouts gives no gain (25.6 vs. 28.4, tuning seeds). Only a full reset helps: +15.0 ± 5.4 over the head reset (5/5 tuning seeds) and +14.6 ± 2.5 on final seeds (10/10). The rule, fixed before running, was to adopt a fix only if it beat the head reset on MinAtar and was not more than 5 points worse on MiniGrid L1; the full reset scored 92.5 vs. 92.5 on L1 and 96.1 vs. 97.1 on C1.

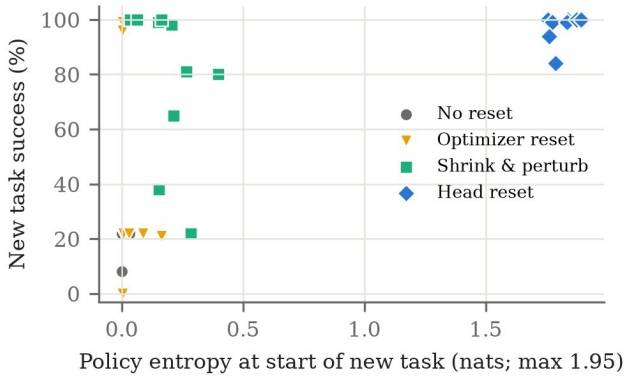


Fig. 4. MiniGrid C1 without retention: policy entropy at the start of the third task vs. its final success (10 seeds per method). A collapsed policy cannot learn; resetting the output heads restores both.

C. Detecting Change vs. Being Told

Reset & Distill [4], told each task end, matches ours on MinAtar (45.1 vs. 44.4, n.s.) but is lower on average on MiniGrid (L1 87.1 vs. 92.7, +5.6 ± 5.5, 15/20; C1 89.4 vs. 96.4, +7.0 ± 5.6, 17/20). It loses an early skill on 2 of 20 seeds in each MiniGrid setup (worst 55%). On seeds 1–10 this happened on 2 of 10 seeds, but on none of the replication seeds, so we weakened this claim. Told distillation drops on the same two C1 seeds (63% and 58%); ours never falls below 89% (L1) or 91% (C1). With our memory, the told reset has no collapses (worst seed 77 and 92), so the memory, not the boundary signal, prevents them.

With the same memory, only the trigger differs: the self-detected reset beats a reset at every given boundary on L1 by 6.6 ± 1.4 points (20/20 seeds; +7.6 ± 2.2 on the replication seeds alone) and ties on C1. Told resets fire at all 7 boundaries of L1, ours only 2–4 times, so it keeps features when the next stage is similar; this is the likely cause, not tested directly.

D. Gradual Change

When each new task is mixed in gradually (MiniGrid: over 200k steps; MinAtar: over the first third of a stage), success may never collapse. On MiniGrid the surprise fired at only 2 of 20 transitions, yet retention was unchanged (97.0 vs. 96.1 abrupt), because the memory alone keeps old skills there. On MinAtar the surprise still fired at 30 of 40 switches, later (median 470k vs. 9k steps), and the final score was unchanged (42.5 vs. 42.7).

E. Order and Sensitivity

With the game order reversed, ours scores 37.2 ± 2.1, CLEAR 24.6 ± 1.9 and no protection 20.0 ± 1.9; ours is higher on 10/10 seeds against both. Order changes the level (games learned first are retained worst). Changing one setting at a time on tuning seeds, mastery 0.7–0.9 and surprise 0.2–0.4 leave the score flat on both domains (MiniGrid L1 91–95, default 92.2; MinAtar 42–47, default 43.4). Memory size matters most: a 4× smaller reservoir costs about 5 points on L1 and 9.0 ± 5.5 on MinAtar (5/5 seeds). On MinAtar, λ trades plasticity for retention (0.1 learns new games at 98% of scratch but keeps less; 1.0 keeps more but learns at 64%).

F. Cost of the Full Reset

When two skills (Unlock, Pickup) are followed by the composite UnlockPickup, final skills are equal to the head-reset variant, but the composite is learned about 2× slower (median 258k vs. 126k steps): the reset discards reusable features.

VII. Limitations

Networks and worlds are small (MiniGrid, MinAtar), with one RL algorithm (PPO) and one set of hyperparameters. The thresholds (0.8, 0.3) and λ were set once on MiniGrid, although sensitivity is low. A slow drift that never drops competence below 0.3 would not trigger a reset (not tested). A full reset discards useful features (Sec. VI-F); choosing what to reset is open. Retention on MinAtar is partial. CLEAR and Reset & Distill are our PPO adaptations, not the original implementations, and joint training is not an upper bound on MinAtar [16].

VIII. Conclusion

An agent told nothing about task changes can, with a small self-triggered loop (remember what you master, distil toward it, restart when the world surprises you), match learners that are told the boundaries and beat CLEAR on two domains, with results replicated on new seeds. Memory carries the past; the reset carries the future. Next, we will ground language instructions in the same continual agent [16].

Reproducibility. Code, configurations, per-seed result files and logs for all runs will be released with the paper.

Acknowledgment

The text of all sections of this paper was generated with the AI system Claude Opus 5.5 [16], working from the authors' experiment reports; the same system also wrote the experiment, analysis and figure code under the authors' direction. The authors reviewed the paper and take full responsibility for its content.

References

[1] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, "Towards continual reinforcement learning: A review and perspectives," J. Artif. Intell. Res., vol. 75, pp. 1401–1476, 2022.

[2] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," Proc. Natl. Acad. Sci., vol. 114, no. 13, pp. 3521–3526, 2017.

[3] J. Schwarz et al., "Progress & compress: A scalable framework for continual learning," in Proc. ICML, PMLR 80, 2018, pp. 4528–4537.

[4] H. Ahn, J. Hyeon, Y. Oh, B. Hwang, and T. Moon, "Reset & distill: A recipe for overcoming negative transfer in continual reinforcement learning," arXiv:2403.05066, 2024.

[5] R. Aljundi, K. Kelchtermans, and T. Tuytelaars, "Task-free continual learning," in Proc. IEEE/CVF CVPR, 2019, pp. 11254–11263.

[6] D. Rao, F. Visin, A. Rusu, R. Pascanu, Y. W. Teh, and R. Hadsell, "Continual unsupervised representation learning," in Proc. NeurIPS, vol. 32, 2019.

[7] S. Kessler, J. Parker-Holder, P. Ball, S. Zohren, and S. J. Roberts, "Same state, different task: Continual reinforcement learning without interference," in Proc. AAAI, 2022.
[8] M. Caccia, J. Mueller, T. Kim, L. Charlin, and R. Fakoor, "Task-agnostic continual reinforcement learning: Gaining insights and overcoming challenges," in Proc. CoLLAs, PMLR 232, 2023, pp. 89–119.
[9] J. Dick, S. Nath, C. Peridis, E. Benjamin, S. Kolouri, and A. Soltoggio, "Statistical context detection for deep lifelong reinforcement learning," in Proc. CoLLAs, 2024.
[10] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in Proc. NeurIPS, vol. 32, 2019, pp. 350–360.
[11] S. Dohare, J. F. Hernandez-Garcia, Q. Lan, P. Rahman, A. R. Mahmood, and R. S. Sutton, "Loss of plasticity in deep continual learning," Nature, vol. 632, pp. 768–774, 2024.
[12] C. Lyle, Z. Zheng, E. Nikishin, B. Avila Pires, R. Pascanu, and W. Dabney, "Understanding plasticity in neural networks," in Proc. ICML, PMLR 202, 2023, pp. 23190–23211.
[13] Z. Abbas, R. Zhao, J. Modayil, A. White, and M. C. Machado, "Loss of plasticity in continual deep reinforcement learning," in Proc. CoLLAs, PMLR 232, 2023, pp. 620–636.
[14] G. Sokar, R. Agarwal, P. S. Castro, and U. Evci, "The dormant neuron phenomenon in deep reinforcement learning," in Proc. ICML, PMLR 202, 2023, pp. 32145–32168.
[15] E. Nikishin, M. Schwarzer, P. D'Oro, P.-L. Bacon, and A. Courville, "The primacy bias in deep reinforcement learning," in Proc. ICML, PMLR 162, 2022, pp. 16828–16847.
[16] Anthropic, "Claude Opus 5.5 (claude-opus-5-5)," large language model, 2026. [Online]. Available: https://www.anthropic.com/claude
[17] M. Wołczyk, M. Zając, R. Pascanu, Ł. Kuciński, and P. Miłoś, "Continual World: A robotic benchmark for continual reinforcement learning," in Proc. NeurIPS, vol. 34, 2021.
[18] Z. Li and D. Hoiem, "Learning without forgetting," in Proc. ECCV, 2016, pp. 614–629.
[19] A. A. Rusu et al., "Policy distillation," in Proc. ICLR, 2016.
[20] D. Isele and A. Cosgun, "Selective experience replay for lifelong learning," in Proc. AAAI, vol. 32, no. 1, 2018.
[21] J. S. Vitter, "Random sampling with a reservoir," ACM Trans. Math. Softw., vol. 11, no. 1, pp. 37–57, 1985.
[22] A. Chaudhry et al., "On tiny episodic memories in continual learning," arXiv:1902.10486, 2019.
[23] J. Ash and R. P. Adams, "On warm-starting neural network training," in Proc. NeurIPS, vol. 33, 2020.
[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
[25] S. Huang et al., "CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms," J. Mach. Learn. Res., vol. 23, no. 274, pp. 1–18, 2022.
[26] M. Chevalier-Boisvert et al., "Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," in Proc. NeurIPS Datasets and Benchmarks Track, 2023.
[27] K. Young and T. Tian, "MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments," arXiv:1903.03176, 2019.